Vibe12 — Edge BACnet & Cloud FDD

py-bacnet-stacks-playground / vibe_code_apps_12

May 25, 2026

Generated May 25, 2026

---

1 Introduction

2 Vibe12 — Edge BACnet & Cloud FDD

End-to-end guide: **Linux gateway** on the BACnet network → **MQTT** to **AWS IoT → DynamoDB → React dashboard** and **Python Rule Lab** for fault detection.

2.1 Start here (checklist)

Field deployment: Master checklist

AI agents & Codex: Agent & Codex getting started — accounts, init workspace, human vs AI limits, cron guardrails.

Phase	Chapter
0	Ben’s office lab (boss Pi)
1	Linux, network & SSH
2	BACnet IoT gateway
3	Commissioning & CSV cleaning
4	AWS IoT Core (PEM, MQTT)
5	Cloud dashboard & FDD
6	Web app features (reference)

2.2 System diagram

```
[Build machine] --SSH/Ansible--> [Gateway Pi on BACnet subnet]
                                      |
                                      | BACnet/IP (BACpypes3)
                                      v
                                 [Controllers / MS/TP router]
                                      |
                                      | MQTT TLS (PEM + key)
                                      v
                                 [AWS IoT Core]
                                      |
                                      v
                                 [Ingest Lambda] --> [DynamoDB]
                                      ^
                                 [Web Lambda] <--- Browser (React
UI + login)
                                 [FDD Lambda]  (every 5 min)
```

2.3 Documentation map (all chapters)

Chapter	Topic
Master checklist	Ordered tasks for integrators
Agent & Codex getting started	Accounts, init, cron guardrails, AI limits
Linux & SSH	Subnet, SSH, firewall
BACnet gateway	Libraries, systemd, deploy
CSV commissioning	Discover → clean → points.csv

Chapter	Topic
AWS IoT	Certificates, topics, policies
Edge deploy	Ansible inventory, dual BACnet
BACnet commissioning	Short commissioning recap
AWS SAM (CloudShell)	tar/zip, upload, deploy
AWS SAM (bensserver)	CLI + `deploy_cloud_from_bensserver.sh`
AI commissioning API	Telemetry flow + BRICK refs for Codex/OpenClaw
Agent workspace	AGENTS.md, MEMORY, skills, BUILD_CHECKPOINTS
Dashboard & FDD	Rule Lab, go-live
Web app features	Per-screen low-level reference
Wire capture	Post-deploy pcap
Commissioning backup	Git backup of CSV
Cloud architecture	Stack outputs
FDD cookbook	Python rule recipes

2.4 PDF bundle

```
cd vibe_code_apps_12
sudo apt install pandoc libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf2.0-0
./scripts/setup_docs_venv.sh
./scripts/build_docs.sh
```

Output: `pdf/vibe12-edge-fdd-guide.pdf`

Chapter order is defined in `docs/manifest.yaml`.

2.5 Tests

```
cd vibe_code_apps_12
python3 -m unittest discover -s tests -v
cd apps/vibe12-web && npm ci && npm test
```

2.6 Repo layout

Path	Role
`ansible/`	SSH deploy, IoT certs, systemd
`edge_bacnet/`	Discover, read driver, MQTT
`commissioning/`	`points.csv` per site/building

Path	Role
`aws_cloud_pipeline/`	SAM, Lambdas, ingest
`apps/vibe12-web/`	React UI (build → `web_lambda/static/app`)
`scripts/`	`build_web_ui.sh`, `build_docs_pdf.py`

3 Master checklist (start here)

4 Vibe12 master checklist

Use this page as a **step-by-step path** from a blank Linux gateway to live fault detection. Each phase links to a detailed chapter. Check boxes as you go.

Who this is for: consulting engineers and integrators who may not write code daily. Technical detail lives in the linked chapters; this page tells you **what to do in order**.

4.1 Phase 0 — Plan

- [ ] You have a **Raspberry Pi or Linux PC** on the **same IP subnet as BACnet** controllers (or a BACnet/IP router such as BASRT-B).
- [ ] You have a **build machine** (bensserver or laptop) with SSH access to the gateway and the Git repo.
- [ ] You have an **AWS account** with permission to use **IoT Core**, **Lambda**, and **DynamoDB** (CloudShell deploy is fine).

Ben’s office lab (boss Pi `192.168.204.12`): edge topics use `site_id` / `building_id` from Ansible (`demo` / `bens-office`), matching SAM `IotTopicPrefix=vibe12` — see Phase 0 — Ben’s office lab.

4.2 Phase 1 — Linux, network, and SSH

Goal: You can log into the gateway from your desk and ping BACnet devices.

- [ ] Gateway has a **static or DHCP-reserved IP** on the BACnet VLAN (example lab: `192.168.204.12`).
- [ ] Your PC or bensserver can **ping** the gateway and the BACnet router/controller.
- [ ] **SSH works** (`ssh user@gateway-ip`). Optional: `ssh-copy-id` so Ansible does not prompt every time.
- [ ] Firewall allows **UDP 47808/47809** (BACnet/IP) on the gateway if you run local BACnet stacks.

→ Details: Linux, network & SSH

4.3 Phase 2 — Deploy the BACnet IoT gateway (edge)

Goal: Software is on the Pi; AWS IoT PEM/key are in place; discover service can run.

- [ ] Edit **Ansible inventory** with `ansible_host` and `ansible_user`.
- [ ] Set **`site_id`** and **`building_id`** in host_vars for this building.
- [ ] Run **`prepare_aws_iot_certs.sh`** on the build machine (once).
- [ ] Run **`deploy.sh`** to copy `edge_bacnet/`, commissioning folder, and systemd units.
- [ ] Confirm units exist: `vibe12-bacnet-discover`, `vibe12-bacnet-read` (read driver when enabled).

→ Details: BACnet IoT gateway · Edge deploy (Ansible)

4.4 Phase 3 — Commission BACnet (CSV cleaning)

Goal: A trimmed **`points.csv`** lists only the points you want in the cloud.

- [ ] Run **discover** (Who-Is / I-Am) → `points_discovered.csv`.
- [ ] **Delete** rows you do not need; set **`enabled=1`** on rows to poll.
- [ ] Fill **`system_id`**, **`brick_class`**, **`brick_tag`** (for FDD and data model later).
- [ ] Save as **`commissioning/.../points.csv`** and back up to Git (`fetch_commissioning.sh`).
- [ ] Redeploy with **`enable_bacnet_read_driver=true`**.

→ Details: Commissioning & CSV cleaning · BACnet commissioning

---

4.5 Phase 4 — AWS IoT Core (certs, policy, MQTT)

Goal: Telemetry leaves the building on MQTT and lands in DynamoDB.

- [ ] IoT **Thing** + certificate (PEM + private key) — prepared by Ansible into `~/vibe_code_apps_12/aws_iot_certs/`.
- [ ] IoT **policy** allows connect as **`basicPubSub`** (lab default) and **publish** to `vibe12/*`.
- [ ] Edge publishes every **60 s** to topic `vibe12/{site_id}/{building_id}/{system_id}/{point_id}/telemetry`
- [ ] Deploy **SAM stack** `vibe12cloud` (ingest + web + scheduled FDD Lambdas).
- [ ] IoT **Rules** forward telemetry to **ingest Lambda**.

→ Details: AWS IoT Core · AWS cloud & SAM

4.6 Phase 5 — Cloud dashboard and data model

Goal: Browser UI shows live points; registry and BRICK model are usable.

- [ ] Open **DashboardUrl** from stack outputs; sign in (engineer login).
- [ ] **Dashboard** — chart and latest values for your site/building.
- [ ] **Data model** — refresh registry; export/import JSON for LLM tagging if needed.
- [ ] Confirm `/api/points/{site}/{building}` lists your series.

→ Details: Web app features · Cloud dashboard & FDD

4.7 Phase 6 — Fault detection (Python rules)

Goal: At least one rule tested and optionally written to the historian.

- [ ] Open **Rule Lab** — pick a default or new rule.
- [ ] **Test rule** on 2–24 h of data (console output, no DB write).
- [ ] **Save draft** → rules stored in DynamoDB.
- [ ] **Write to database (go-live)** → FDD backfill + status row for dashboard analytics.
- [ ] Optional: **BRICK scope** — run same rule across all matching point classes.

→ Details: Cloud dashboard & FDD · FDD rule cookbook

4.8 Phase 7 — Optional diagnostics

- [ ]

- [ ] **Wire capture:** deploy with `--pcap` → `bacnet.pcap` for Wireshark (wire capture).
- [ ] **PDF guide:** `./scripts/build_docs.sh` on Linux with pandoc (PDF bundle).

4.9 Quick reference

Item	Typical value (lab)
Gateway IP	`192.168.204.12`
BACnet router (MS/TP)	`192.168.204.200`
Site / building	`demo` / `pi` or `bens-office`
MQTT client id	`basicPubSub`
Poll interval	60 s
Cloud stack	`vibe12cloud` (us-east-2)

When every box in phases 1–6 is checked, you have a complete **edge → cloud → FDD** loop.

5 Agent & Codex getting started

6 Agent and Codex getting started

This guide is for **integrators and AI agents** using `vibe12_agent_spec/` after cloning the repository. Product code lives in `vibe_code_apps_12/`; **agent memory is local to each clone** and is not committed to Git.

6.1 Prerequisites checklist

6.1.1 Accounts and tools

- [ ] **Git** — clone this repository on a build machine with network access to your gateway and AWS.

- [ ] **OpenAI Codex CLI** — installed (`codex --version`); logged in (`codex login`).
- [ ] **AWS account** — permissions for IoT Core, Lambda, DynamoDB, CloudFormation (SAM deploy).
- [ ] **AWS CLI v2** and **SAM CLI** on the build machine (see AWS deploy from bensserver).
- [ ] **SSH** to the edge gateway (keys or password — human-managed).
- [ ] **Python 3.10+** and **Node.js** (for tests and web build).

6.1.2 After clone — initialize local agent workspace

```
cd vibe_code_apps_12
vibe12_agent_spec/bin/vibe12_workspace_init.sh
```

This creates **local-only** files from `vibe12_agent_spec/templates/`:

File	Purpose
`MEMORY.md`	Curated standing brief for your project
`BUILD_CHECKPOINTS.md`	Sprint queue (critique updates **Next for mini**)
`memory/commissioning/PHASE_NOTEPAD.md`	Site bind, devices, URLs
`cron_codex/.env`	Models, wake limits, paths
`cron_codex/state/operator_notes.md`	Your notes to the agent

Fill **PHASE_NOTEPAD** and **MEMORY.md** with your site facts. Do not commit them.

6.2 Human vs AI responsibilities

Task	Human	AI agent (Codex / Cursor)
AWS root / billing / IAM user creation	**Required**	Never
IoT certificates	**Required**	May prepare files; human deploys

Task	Human	AI agent (Codex / Cursor)
and policy approval		
SSH credentials and gateway access	**Required**	Never invent passwords or keys
BACnet **write** commands to field devices	**Sign-off required**	Read-only by default
`points.csv` enable rows	**Approves**	May discover and propose rows
`samconfig.toml` passwords	**Sets locally**	Never commit
Cloud deploy to production	**Approves**	May run scripts when asked
BRICK / SparkQL semantic validation	**Sign-off**	May draft graphs and rules
Scheduled Codex cron	**Installs with `--yes`**	May suggest schedule; must not bypass guardrails
Physical wiring and controller config	**Required**	Never

6.3 Interactive Codex (recommended first)

```
cd vibe_code_apps_12
vibe12_agent_spec/bin/vibe12_codex_tui.py
```

Command	Model	Purpose
(normal prompt)	gpt-5.4-mini	One implementation slice
`/critique`	gpt-5.5	Review + rewrite **Next for mini (ordered)**
`/wake 1`	mini + critique	One cheap orchestrated cycle
`/new`	—	Fresh mini session

On Linux hosts where bubblewrap fails (`RTM_NEWADDR`), the TUI auto-bypasses the Codex sandbox so shell tools work.

6.4 Orchestrated wakes (mini → critique)

```
gpt-5.4-mini  →  executes BUILD_CHECKPOINTS "Next for mini (ordered)"
gpt-5.5       →  rewrites that queue for the next wake
```

```
MINI_INVOCATIONS_PER_WAKE=1 vibe12_agent_spec/cron_codex/bin/vibe12_wake.sh
```

Context export each wake: `cron_codex/state/context_since_last_wake.md` (operator notes + PHASE_NOTEPAD).

6.5 Scheduled cron (optional)

Do not enable until interactive wakes behave well.

Guardrails (defaults in `cron_codex/env.example`):

Guardrail	Default	Purpose
`MIN_MINUTES_BETWEEN_WAKES`	120	Debounce overlapping runs
`MAX_WAKES_PER_DAY`	12	Daily spend cap
`MINI_INVOCATIONS_PER_WAKE`	3 (max 5)	Limit minis per wake
`flock` lock	`/tmp/vibe12_codex_wake.lock`	One wake at a time
`waiting_human` file	—	Pause all wakes
`DONE_AUTOMATION` file	—	Silent exit until removed
Install requires `--yes`	—	Prevents accidental cron

```
# Human must run explicitly:
vibe12_agent_spec/cron_codex/bin/vibe12_install_cron.sh --yes
```

```
# Pause
touch vibe12_agent_spec/cron_codex/state/waiting_human

# Remove cron
vibe12_agent_spec/cron_codex/bin/vibe12_remove_cron.sh
```

Agents **must not** set `VIBE12_CRON_ALLOW_AGGRESSIVE=true` or install cron without human approval.

6.6 Smoke before claiming “done”

```
./scripts/validate_cloud_pipeline.sh
python3 -m unittest discover -s tests -q
```

Commissioning API (replace site/building):

```
GET /api/commissioning/status/{site}/{building}
```

6.7 Further reading

- Master checklist (field deploy)
- Agent workspace README
- Cron orchestration
- GUARDRAILS

7 Linux, network & SSH

8 Linux, network & SSH

Before BACnet or AWS, the **gateway computer** must sit on the **same network as your controllers** and accept **SSH** from your build machine.

8.1 What you need

Item	Why
Raspberry Pi 4/5 or Linux PC	Runs BACnet discover/read and MQTT client
Ethernet (preferred) or Wi-Fi	Stable link to BACnet VLAN
Static or reserved DHCP IP	

Item	Why
	Ansible and bookmarks do not break
SSH enabled	Ansible deploys without a keyboard on the Pi

8.2 Network layout (typical lab)

```
[Your PC / bensserver] ----SSH----> [Gateway Pi 192.168.204.12]
                                          |
                                          | UDP BACnet/IP
                                          v
                                  [BASRT-B router 192.168.204.200]
                                          |
                                          MS/TP
                                          v
                                  [Field controller device 5007]
```

8.3 Step-by-step

8.3.1 1. Put the gateway on the BACnet subnet

- Configure **IPv4 address**, **mask**, and **gateway** so the Pi can reach:
  - Your BACnet/IP router (if MS/TP is behind a router)
  - Any BACnet devices that speak IP directly
- Example lab subnet: `192.168.204.0/24`.

Check: From the Pi, `ping 192.168.204.200` (router) succeeds.

8.3.2 2. Enable SSH

On Raspberry Pi OS:

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

Create a user (e.g. `ben`) with sudo for Ansible `become`.

8.3.3 3. Log in from your build machine

```
ssh ben@192.168.204.12
```

If you use passwords, install **`sshpass`** on the build machine for Ansible:

```
sudo apt install sshpass
```

8.3.4 4. Optional — passwordless SSH

```
ssh-copy-id ben@192.168.204.12
```

Then Ansible `./deploy.sh -v` runs without `-ask-pass`.

8.3.5 5. Firewall notes

- **Outbound HTTPS (443)** — required for **AWS IoT Core** MQTT.
- **UDP 47808** — default BACnet/IP (PiTemp / some tools).
- **UDP 47809** — Vibe12 read driver when dual-stack on boss Pi (avoids port clash).

8.4 Inventory entry

In `ansible/inventory.yml`:

```yaml
bacnet_pi:
  ansible_host: 192.168.204.12
  ansible_user: ben
```

8.5 Troubleshooting

Symptom	What to check
SSH timeout	Wrong IP, VLAN, or firewall
Permission denied	User name, key, or password
BACnet discover finds nothing	Pi not on same subnet as router; wrong `--address` bind
MQTT fails later	Outbound 443 blocked

Next: BACnet IoT gateway.

9 BACnet IoT gateway

10 BACnet IoT gateway

The **edge gateway** is a small Linux service set that discovers BACnet points, polls them on a schedule, and publishes JSON to **AWS IoT Core** over MQTT.

10.1 Software stack (what runs on the Pi)

Component	Role
`edge_bacnet/`	Python scripts: discover, read driver, MQTT publish
BACpypes3	BACnet/IP (and route-aware MS/TP via router)
systemd units	`vibe12-bacnet-discover`, `vibe12-bacnet-read`
AWS IoT device SDK	MQTT TLS to `*.iot.*.amazonaws.com`
Ansible	Copies code + certs; no `git clone` on the Pi

10.2 How data flows

```
points.csv  -->  read driver (60 s)  -->  RPM read present-value
       |                                        |
       |                                        v
       +---------------------------->  MQTT publish
                                         vibe12/site/bld/sys/
point/telemetry
```

Discover is **read-only** (Who-Is / I-Am, optional RPM for metadata). The read driver only polls rows with **`enabled=1`** in `points.csv`.

10.3 Deployment (Ansible)

From `vibe_code_apps_12/ansible` on your build machine:

```
./prepare_aws_iot_certs.sh          # once: PEM + key into ansible/files/aws_iot/
./deploy.sh --limit bacnet_pi \
  -e enable_bacnet_read_driver=true \
  --ask-pass --ask-become-pass -v
```

Files land under `~/vibe_code_apps_12/` on the gateway:

```
edge_bacnet/          # Python modules
commissioning/        # points.csv per site/building
aws_iot_certs/        # device.pem, private.key, AmazonRootCA1.pem
scripts/              # e.g. bacnet_tcpdump_once.sh
```

10.4 Libraries and Python environment

- Repo root **`requirements.txt`** — BACnet + MQTT dependencies (installed by Ansible on the Pi).
- Read driver invoked as: `python3 -m edge_bacnet.read_driver` (exact module path in systemd template).

10.5 BACnet bind and MS/TP router

In **`host_vars/bacnet_pi.yml`** (example):

```
site_id: demo
building_id: bens-office   # must match commissioning/ and cloud dashboard picker
bacnet_edge_bind_address: "192.168.204.12/24:47809"
bacnet_route_aware: true
bacnet_router_ip: 192.168.204.200
bacnet_mstp_net: 2000
```

Route-aware mode sends Who-Is through the BACnet/IP router to MS/TP devices.

10.6 Dual BACnet on boss Pi (lab)

Service	UDP port	Purpose
`bacnet-ds18b20`	47808	Local 1-wire demo
`vibe12-bacnet-read`	47809	Commissioned points + GPIO in same MQTT cycle

Only **one MQTT client id** (`basicPubSub`) is used — GPIO samples ride with BACnet in the read driver when enabled.

10.7 Verify after deploy

```
        ssh ben@192.168.204.12 'systemctl is-active vibe12-bacnet-read'
        ssh ben@192.168.204.12 'journalctl -u vibe12-bacnet-read -n 20 --no-pager'
```

Look for `published N samples` and no `NOT_AUTHORIZED` MQTT errors.

Next: Commissioning & CSV cleaning.

11 Commissioning & CSV cleaning

12 Commissioning & CSV cleaning

Commissioning turns a raw BACnet scan into a **clean point list** the cloud can understand. Think of it as **editing a spreadsheet** — no programming required.

12.1 Files you will touch

File	When
`points_discovered.csv`	Auto-generated by discover (do not hand-edit as source of truth)
`points.csv`	**Your** trimmed list — read driver reads this
`commissioning/{site}/{building}/points.csv`	Git-backed path on bensserver

12.2 Phase A — Discover (automatic)

On the gateway:

```
cd ~/vibe_code_apps_12
sudo systemctl start vibe12-bacnet-discover
journalctl -u vibe12-bacnet-discover -n 80 --no-pager
```

Or run discover via Ansible-deployed one-shot. Output columns typically include device id, object type, instance, present value, and suggested tags.

12.3 Phase B — Clean the CSV (manual)

Open `points_discovered.csv` in Excel, LibreOffice, or a text editor.

Do this for every row you keep:

1. Set **`enabled`** to `1` (only enabled rows are polled).
2. Set **`system_id`** — equipment group (e.g. `office`, `ahu-1`, `vav-3`).
3. Set **`brick_class`** — BRICK point type (e.g. `Zone_Air_Temperature_Sensor`).
4. Set **`brick_tag`** — short operator label (e.g. `ZAT`, `SAT`).
5. Delete rows for devices you **do not** want in the cloud.

Delete or disable:

- Duplicate objects
- Internal/debug points
- Points on devices you do not own

Save the result as **`points.csv`** in the commissioning folder for your site/building.

12.4 Phase C — Back up to Git (recommended)

From bensserver:

```
cd vibe_code_apps_12/ansible
./fetch_commissioning.sh --limit bacnet_pi -v
git add ../commissioning/
git commit -m "Commission points for demo/bens-office"
```

12.5 Phase D — Enable polling

```
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true -v
```

12.6 Column reference (typical)

Column	Meaning
`enabled`	`1` = poll and publish; `0` = ignore
`system_id`	Equipment / system name for grouping
`point_id`	Stable id used in MQTT topic segment
`brick_class`	BRICK type for FDD rules and data model
`brick_tag`	Human label
`device_instance`	BACnet device id
`object_type` / `object_instance`	BACnet object

Exact headers match your discover template — if a column is missing, discover again after updating `edge_bacnet`.

12.7 Checklist

- [ ] Discover completed without BACnet errors in journal
- [ ] `points.csv` has only intended rows with `enabled=1`
- [ ] Every enabled row has `system_id` and `brick_class`
- [ ] File committed or backed up off the Pi
- [ ] Read driver journal shows publish count matching enabled points

Next: AWS IoT Core.

13 AWS IoT Core (PEM, MQTT)

14 AWS IoT Core

AWS IoT Core is the **MQTT broker in the cloud**. The gateway connects with a **certificate (PEM)** and publishes telemetry; **IoT Rules** invoke **Lambda** to write **DynamoDB**.

14.1 What you need in AWS

Resource	Purpose
IoT Thing	Logical name for your gateway (e.g. `bosspi`)
Device certificate	`device.pem.crt` + `private.key` + Amazon root CA
IoT policy	Allows `connect`, `publish`, `subscribe` for your topics
IoT Rules	SQL on topics → trigger **ingest Lambda**
Lambda + DynamoDB	Deployed by SAM stack `vibe12cloud`

Edge certs are **not** stored in Git. Ansible copies them from `ansible/files/aws_iot/` after you run `prepare_aws_iot_certs.sh` on the build machine.

14.2 Certificate files (on the Pi)

After deploy:

```
~/vibe_code_apps_12/aws_iot_certs/
  AmazonRootCA1.pem      # or your region root CA
  device.pem.crt         # device certificate (PEM)
  private.key            # private key — keep secret
```

The read driver uses these paths in environment variables set by systemd (from Ansible templates).

14.3 IoT policy (lab default)

The lab certificate is often registered for MQTT client id **`basicPubSub`**. Policy must allow at least:

- **Connect** with client id `basicPubSub`
- **Publish** to `vibe12/#` (multi-level; not `vibe12/*` which is only one segment)
- **Subscribe** if you use command topics (optional)

Example policy fragment lives in `aws_iot_core_test/policy-vibe12-multi-client.json` in the repo.

Symptom: `NOT_AUTHORIZED` in `journalctl -u vibe12-bacnet-read` → policy or wrong client id.

14.4 MQTT topic layout

Every telemetry message uses:

```
vibe12/{site_id}/{building_id}/{system_id}/{point_id}/telemetry
```

Example:

```
vibe12/demo/pi/office/digital-temp-degF/telemetry
```

Payload (JSON) includes:

- `series_id` — often `site#building#system#point` (DynamoDB key)
- `value`, `unit`, `ts` / `ts_ms`
- `brick_class`, `brick_tag`, `site_id`, `building_id`, …

14.5 IoT Rule (in SAM template)

Rule	SQL (concept)	Action
BACnet telemetry	`vibe12/+/+/+/+/telemetry`	ingest Lambda

Rule injects `topic()` as `mqtt_topic` in the Lambda event for parsing.

14.6 Prepare certs on build machine

```
cd vibe_code_apps_12/ansible
./prepare_aws_iot_certs.sh
```

Follow script prompts (AWS CLI must be configured). Then deploy to the Pi so files appear under `aws_iot_certs/`.

14.7 Verify MQTT without the dashboard

1. AWS Console → **IoT Core** → **MQTT test client** → subscribe to `vibe12/#`
2. Confirm messages every 60 s after read driver is active
3. CloudWatch → **ingest Lambda** → invocations increasing

14.8 Security checklist

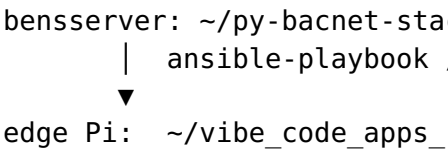
- ☐ Private key never committed to Git
- ☐ Policy is least-privilege (not `iot:*` in production)
- ☐ Certificate rotation plan documented for customer handoff

Next: deploy cloud stack in AWS cloud & SAM, then Cloud dashboard & FDD.

15 Edge deploy (Ansible reference)

16 Edge deploy (Ansible)

Ansible pushes the Vibe12 stack from your **build machine** (bensserver) to edge hosts over **SSH**. The Pi does **not** need a git clone — only selected files are copied.



Beginner walkthrough: `ansible/ANSIBLE-BEGINNER.md`.

16.1 Two edge roles

Role	Example host	GPIO	MQTT
Building gateway	`tower_a_edge`	Off	

Role	Example host	GPIO	MQTT
			BACnet read driver only
Boss Pi test bench	`bacnet_pi` @ `192.168.204.12`	DS18B20 + BACnet MS/TP	One `basicPubSub` client (GPIO + BACnet)

16.2 Quick start (boss Pi)

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12/ansible
./prepare_aws_iot_certs.sh    # once on bensserver
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --ask-pass --ask-become-pass -v
```

Optional 5-minute BACnet wire capture:

```
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --pcap --ask-pass --ask-become-pass -v
```

16.3 Inventory

Edit `ansible/inventory.yml`:

```
bacnet_pi:
  ansible_host: 192.168.204.12
  ansible_user: ben
```

Per-building MQTT scope: `ansible/host_vars/<host>.yml` — `site_id`, `building_id`.

16.4 AWS IoT certificate (shared across edges)

1. Run `./prepare_aws_iot_certs.sh` → `ansible/files/aws_iot/`
2. Deploy copies PEM/key to `~/vibe_code_apps_12/aws_iot_certs/` on each host
3. IoT **policy** must allow publish to `vibe12/*` and connect as **`basicPubSub`** (current lab cert) or extend policy per `aws_iot_core_test/policy-vibe12-multi-client.json`

16.5 Common commands

Goal	Command
Deploy one host	`./deploy.sh --limit HOST -v`
Enable BACnet polling	`-e enable_bacnet_read_driver=true`
Backup commissioned CSV	`./fetch_commissioning.sh --limit HOST -v`
Deploy + 5 min pcap	`--pcap`
Verify only	`./deploy.sh --verify`

16.6 Dual BACnet stacks (boss Pi)

- **PiTemp** — `bacnet-ds18b20.service`, UDP **47808**
- **Vibe12Edge** — `vibe12-bacnet-read.service`, UDP **47809**

Both can run together; MQTT publishes from the read driver only.

16.7 Verify on edge

```
systemctl is-active vibe12-bacnet-read
journalctl -u vibe12-bacnet-read -n 20 --no-pager
```

MQTT test client (AWS console): `vibe12/demo/bens-office/#`

17 BACnet commissioning (quick)

18 BACnet commissioning

Commission BACnet points on the edge gateway, trim the CSV, enable the read driver, then confirm MQTT on AWS IoT.

18.1 Prerequisites

- Validated BACnet bind (`--address IP/prefix` or `bacnet_edge_bind_address` in host_vars)
- AWS IoT cert on bensserver: `ansible/prepare_aws_iot_certs.sh`
- `site_id` and `building_id` in **host_vars** per building

- Lab cert: MQTT client **basicPubSub**, publish `vibe12/*`

18.2 Phase 1 — Discover (read-only)

On the edge:

```
cd ~/vibe_code_apps_12
sudo systemctl start vibe12-bacnet-discover
journalctl -u vibe12-bacnet-discover -n 80 --no-pager
```

Output: `points_discovered.csv`.

MS/TP via BASRT-B (boss Pi example in `host_vars/bacnet_pi.yml`):

```
bacnet_route_aware: true
bacnet_router_ip: 192.168.204.200
bacnet_mstp_net: 2000
bacnet_discover_range_low: 5007
bacnet_discover_range_high: 5007
```

18.3 Phase 2 — Edit CSV

1. Open `points_discovered.csv`
2. Delete unwanted rows; set **`enabled=1`** on points to poll
3. Fill **`system_id`**, **`brick_class`**, **`brick_tag`**
4. Save as **`points.csv`**

Git backup from bensserver:

```
cd vibe_code_apps_12/ansible
./fetch_commissioning.sh --limit bacnet_pi -v
git add ../commissioning/
git commit -m "Commission BACnet points for demo/bens-office"
```

18.4 Phase 3 — Enable read driver

```
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true -v
```

Verify:

```
ssh ben@192.168.204.12 'journalctl -u vibe12-bacnet-read -n 15 --no-pager'
```

Expect: `published 6 samples` on boss Pi (4 BACnet + 2 GPIO) every 60 s.

MQTT topic pattern:

`vibe12/{site_id}/{building_id}/{system_id}/{point_id}/telemetry`

18.5 Phase 4 — Cloud

Deploy SAM stack — see AWS cloud & SAM deploy.

Smoke:

```
curl -sS "${URL}/api/buildings"
curl -sS "${URL}/api/points/demo/bens-office"
```

18.6 Rollout checklist

- [ ] Discovery CSV exported
- [ ] `points.csv` trimmed; Brick tags filled
- [ ] Read driver publishing on `vibe12/…/telemetry`
- [ ] Cloud ingest shows points in dashboard
- [ ] One FDD rule tested in Rule Lab

18.7 Safety

- Read-only BACnet — no writes until production sign-off
- Never commit PEM/private keys to Git

19 AWS cloud & SAM deploy

20 AWS cloud & SAM deploy

Deploy the **vibe12cloud** stack (IoT rules → DynamoDB → dashboard Lambdas) in **us-east-2**.

Two paths:

Path	Guide
B — bensserver (recommended if you	Deploy SAM from bensserver — `./scripts/`

Path	Guide
have AWS CLI keys here)	`deploy_cloud_from_bensserver.sh`, no tarball
CloudShell	This page (upload tar from Windows)

> **CloudShell upload:** AWS Console → **CloudShell** → **Actions** → **Upload file** (not the IoT Core console).

Prerequisites: Pi (or edge) publishing to AWS IoT; `aws sts get-caller-identity` works in CloudShell.

20.1 Copy-paste checklist (full path)

Step	Where	What
A0	bensserver	**Build React UI →** `web_lambda/static/app/` (`./scripts/build_web_ui.sh`)
A	bensserver	`tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz …` (includes built UI, ~3–8 MB)
B	Windows	Download `.tar.gz` or `.zip` from bensserver (SCP / VS Code)
C	CloudShell	`rm -f` old archive + `rm -rf ~/aws_cloud_pipeline`
D	CloudShell UI	**Actions → Upload file**
E	CloudShell	`tar -xzf …` or `unzip`
F	CloudShell	`cp samconfig.toml.example samconfig.toml` → **set WebPassword + AuthSecret**
G	CloudShell	`sam build` → `sam deploy --force-upload`
H	CloudShell	`curl /api/health` + login test (or `./scripts/verify_cloud_dashboard.sh`)
I	Browser	Open **DashboardUrl** → sign in (`engineer` + your password)

Same content also lives in `aws_cloud_pipeline/README.md` (longer notes) and `DEPLOYED.md` (URLs after deploy).

20.2 What is SAM?

SAM packages `aws_cloud_pipeline/template.yaml` and runs **`sam deploy`** (CloudFormation). Stack name: **`vibe12cloud`**.

20.3 Step A0 — Build the React UI on bensserver (required)

The dashboard is a **Vite/React** app baked into the Lambda zip at `web_lambda/static/app/`. Run this **before** packing the tarball.

Node.js must be **≥ 20.19** (or 22). Check: `node -v`.

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12

# Optional: run unit tests first
cd apps/vibe12-web && npm ci && npm test && cd ../..

# Build and copy dist → aws_cloud_pipeline/web_lambda/static/app/
./scripts/build_web_ui.sh

# Confirm UI is present (~5 MB uncompressed)
ls -lh aws_cloud_pipeline/web_lambda/static/app/index.html
du -sh aws_cloud_pipeline/web_lambda/static/app
```

If you skip this step, the cloud site may show the old HTML dashboard or `spa not built` errors.

20.4 Step A — Pack on bensserver (Linux)

20.4.1 Option 1 — `.tar.gz` (recommended)

```
cd ~/py-bacnet-stacks-playground
tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz \
  -C vibe_code_apps_12 aws_cloud_pipeline
ls -lh ~/vibe12-aws-cloud-pipeline.tar.gz
```

Expect **~3–8 MB** (includes React `static/app/`). If the file is ~20 bytes or only ~100 KB, the pack failed or **A0** was skipped.

Size check on bensserver:

```
test -f ~/vibe12-aws-cloud-pipeline.tar.gz && \
  test $(stat -c%s ~/vibe12-aws-cloud-pipeline.tar.gz) -gt
1000000 && \
```

```
          echo "OK: tarball ready" || echo "ABORT: too small — run
build_web_ui.sh and re-tar"
```

With unit tests (optional):

```
        tar -czf ~/vibe12-aws-cloud-pipeline.tar.gz \
          -C vibe_code_apps_12 aws_cloud_pipeline tests
```

20.4.2 Option 2 — `.zip` (Windows-friendly)

On bensserver:

```
        cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
        zip -r ~/vibe12-aws-cloud-pipeline.zip aws_cloud_pipeline
        ls -lh ~/vibe12-aws-cloud-pipeline.zip
```

On Windows (PowerShell), from a folder that contains `aws_cloud_pipeline`:

```
        Compress-Archive -Path aws_cloud_pipeline -DestinationPath vibe12-
aws-cloud-pipeline.zip
```

Upload **either** `.tar.gz` or `.zip` to CloudShell (see below). For **zip**, extract with `unzip` instead of `tar`.

20.5 Step B — Download to Windows (optional)

Copy from bensserver to your PC (SCP, SFTP, or VS Code remote). You will upload this file through the **AWS Console**, not by pasting into the shell.

20.6 Step C — CloudShell: clean home before upload

Region: **us-east-2** → open **CloudShell**.

```
        rm -f ~/vibe12-aws-cloud-pipeline.tar.gz ~/vibe12-aws-cloud-
pipeline.zip
        rm -rf ~/aws_cloud_pipeline ~/vibe_code_apps_12
        ls ~
```

CloudShell does **not** overwrite an existing upload with the same filename — delete first.

20.7 Step D — Upload from Windows

1. AWS Console → **CloudShell** (us-east-2)
2. **Actions → Upload file**
3. Select `vibe12-aws-cloud-pipeline.tar.gz` (or `.zip`)

Confirm size:

```
ls -lh ~/vibe12-aws-cloud-pipeline.tar.gz
test -f ~/vibe12-aws-cloud-pipeline.tar.gz && \
  test $(stat -c%s ~/vibe12-aws-cloud-pipeline.tar.gz) -gt
1000000 && \
  echo "OK: tarball ready" || echo "ABORT: re-upload (expect ~3–8
MB with React UI)"
```

For zip:

```
ls -lh ~/vibe12-aws-cloud-pipeline.zip
```

20.8 Step E — Extract

Tar:

```
cd ~
tar -xzf ~/vibe12-aws-cloud-pipeline.tar.gz
cd ~/aws_cloud_pipeline
ls
```

You should see `template.yaml`, `samconfig.toml.example`, `ingest_lambda/`, `web_lambda/`, `fdd_lambda/`, and **`web_lambda/static/app/index.html`** (React UI).

Zip:

```
cd ~
unzip -o ~/vibe12-aws-cloud-pipeline.zip -d ~
cd ~/aws_cloud_pipeline
```

20.9 Step F — `samconfig.toml` + login secrets

On bensserver (recommended once, before packing the tar):

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12/
aws_cloud_pipeline
```

```
cp samconfig.toml.example samconfig.toml
nano samconfig.toml
```

Edit only these two placeholders in `parameter_overrides`:

Parameter	Set to
`WebPassword`	Replace `REPLACE_WITH_YOUR_PASSWORD`
`AuthSecret`	Replace `REPLACE_WITH_LONG_RANDOM_SECRET_MIN_32_CHARS`

Optional: bump `DeployRevision` each deploy (`"5"`, `"6"`, …).

`samconfig.toml` is **gitignored** — safe on bensserver, included in your tarball, never pushed to GitHub.

On CloudShell (if tarball already contains your edited `samconfig.toml`):

```
cd ~/aws_cloud_pipeline
ls samconfig.toml    # should exist — skip cp/nano
```

If missing:

```
cp samconfig.toml.example samconfig.toml
nano samconfig.toml
```

Do **not** use `sam deploy --guided` after `samconfig.toml` exists.

20.10 Step G — Build and deploy

```
cd ~/aws_cloud_pipeline
rm -rf .aws-sam
sam build --no-cached
sam validate --lint
sam deploy --force-upload
```

Hierarchical BACnet ingest rule is included: `vibe12/+/+/+/+/telemetry`.

20.11 Step H — Verify (CLI)

```
export AWS_REGION=us-east-2
export STACK=vibe12cloud
URL=$(aws cloudformation describe-stacks --stack-name "$STACK" --region "$AWS_REGION" \
  --query "Stacks[0].Outputs[?OutputKey=='DashboardUrl'].OutputValue" --output text | tr -d
```

```
'\n\r')
        URL="${URL%/}"
        echo "Dashboard: $URL"

        # Health (no login required)
        curl -sS "${URL}/api/health" | python3 -m json.tool

        # Login API (use the same WebPassword you set in samconfig.toml)
        curl -sS -X POST "${URL}/api/auth/login" \
          -H 'Content-Type: application/json' \
          -d
'{"username":"engineer","password":"YOUR_WEB_PASSWORD_HERE"}' |
python3 -m json.tool
```

Expect login JSON with `"ok": true` and a `"token"` field.

SPA check: open `$URL` in a browser — you should see **Vibe12 Cloud** sign-in (not the old tabbed HTML). After login: Dashboard, Rule Lab, Data Model, System in the sidebar.

IoT test client: subscribe to `vibe12/#` (or `vibe12/{site_id}/{building_id}/#` for one building).

20.12 Step I — Browser

1. Open the **Dashboard** URL from step H.
2. Sign in as **`engineer`** (or your `WebUsername`) with **`WebPassword`** from `samconfig.toml`.
3. Pick **site / building** in the top bar (must match edge MQTT `site_id` / `building_id`).
4. **Dashboard** — chart; **Rule Lab** — FDD Python; **Data model** — registry + import.

Troubleshooting login: Browser devtools → Console — filter `vibe12`. Failed login shows `[vibe12][api] 401`. Add `?log=debug` to the URL for more API timing lines (not noisy by default).

20.13 Tear down

```
        sam delete --stack-name vibe12cloud
```

20.14 Troubleshooting

Problem	Fix
Upload seems ignored	`rm -f ~/vibe12-aws-cloud-pipeline.tar.gz` then re-upload
`Missing option '--stack-name'`	`cp samconfig.toml.example samconfig.toml`
Tarball missing in CloudShell	Confirm `ls -lh` shows **> 1 MB** before extract (with React UI, ~3–8 MB)
Old HTML dashboard after deploy	Re-run **A0** `build_web_ui.sh` on bensserver, re-tar, re-deploy
Login fails / 401	`WebPassword` in `samconfig.toml` must match what you type; redeploy after changing params
`spa not built` error	`web_lambda/static/app/index.html` missing from tarball — run A0
Stack name	Use **`vibe12cloud`** only (no hyphens in some resource names)

More detail: DEPLOYED.md (stack outputs, example URLs).

21 Cloud dashboard & FDD (Python)

22 Cloud dashboard & FDD (Python)

After telemetry is in **DynamoDB**, the **web Lambda** serves a React dashboard and **Rule Lab** where you author **Python** fault rules (Bake-a-Py style).

22.1 Deploy the cloud stack

See AWS cloud & SAM deploy. Summary:

1. Pack `aws_cloud_pipeline/` on bensserver (`tar` or `zip`).
2. Upload to **CloudShell** (us-east-2).
3. `sam build` → `sam deploy --stack-name vibe12cloud`.
4. Note **DashboardUrl** and set **WebPassword** / **AuthSecret** parameters.

Before deploy, build the UI on bensserver (see AWS cloud & SAM deploy **Step A0**):

```
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
./scripts/build_web_ui.sh
```

This copies the React app into `web_lambda/static/app/` (Lambda-only hosting, no S3). Then pack `aws_cloud_pipeline` into the tarball and deploy from CloudShell as usual.

22.2 Sign in

Open **DashboardUrl** in a browser. Log in with SAM parameters **`WebUsername`** / **`WebPassword`** (default user `engineer`).

API calls send `Authorization: Bearer <token>`. Static JS is public; **API routes require a valid token** when auth env vars are set.

22.3 Rule Lab workflow (non-technical summary)

Button	What it does
Test rule	Runs Python on recent data; shows results in console — **does not** change production DB
Save draft	Stores rules in DynamoDB for next visit
Write to database	Saves rules + runs **7-day backfill** in chunks; updates FDD status for dashboard

22.4 Python rule shape

Each rule has:

- **Name** — display only (rename with ✎).

- **Config** — numbers like bounds, tolerances, rolling average minutes.
- **Code** — `def evaluate(row, cfg, prev_row=None, rows=None):` return `True` to flag a row.

Temperature helpers: `row["temp"]`, `cfg["bounds_low"]`, optional `row["temp_rolling_avg"]`.

Full recipes: FDD expression rule cookbook.

22.5 BRICK-scoped rules

If your registry has **BRICK classes**, Rule Lab can run one rule across **all matching points** (equipment type + point class). Requires telemetry and optional data model import.

22.6 Scheduled FDD

A separate **FDD Lambda** runs every **5 minutes** on recent data using saved rules (same engine as go-live, smaller window).

22.7 Smoke tests (copy-paste)

Replace `URL` with your Function URL:

```
curl -sS "$URL/api/health" | head
curl -sS -X POST "$URL/api/auth/login" \
  -H 'Content-Type: application/json' \
  -d '{"username":"engineer","password":"YOUR_PASSWORD"}'
```

22.8 Checklist

- [ ] `./scripts/build_web_ui.sh` run before `sam deploy`
- [ ] Dashboard shows latest temperature
- [ ] Points API lists commissioned series
- [ ] One rule **Test** succeeds
- [ ] **Save draft** + **Write to database** complete without 502 (use shorter test window if timeout)

Next: Web app features (low-level).

23 Web app features (reference)

24 Web app features (reference)

Low-level description of each screen in **Vibe12 Cloud** (`apps/vibe12-web`), served from the **web Lambda** Function URL.

24.1 Architecture

```
Browser (React SPA)
    |  HTTPS same origin
    v
Lambda Function URL
    |-- GET /assets/*     static JS/CSS (Vite build)
    |-- GET /             index.html (SPA)
    |-- POST /api/auth/login
    |-- GET/POST /api/*   Python (DynamoDB, FDD engine)
```

Stack: **React 19**, **TypeScript**, **Vite**, **Plotly**, **CodeMirror** (Python), styling aligned with **Open-FDD desktop-ui**.

24.2 Login

Item	Detail
Endpoint	`POST /api/auth/login` with `{username, password}`
Session	Bearer token in `sessionStorage`
Config	Lambda env `VIBE12_WEB_USER`, `VIBE12_WEB_PASSWORD`, `VIBE12_AUTH_SECRET`
Troubleshooting	Browser console prefix `[vibe12] [api]`; add `?log=debug` for timing lines

24.3 Dashboard

Feature	Behavior
Site / building	Top bar; drives all API queries
Latest °C / °F	From last point in readings response
FDD status chip	From `fdd_open` summary (after go-live)
History	6–168 h lookback
Display unit	Imperial °F or metric °C for chart axis
Rolling average	1–15 min server-side average series
Chart	Plotly line trace; auto-refresh ~30 s (silent)
API	`GET /api/readings?hours=&rolling_avg_minutes=&temp_unit=`

24.4 Rule Lab

Feature	Behavior
Rule list	Dropdown of all rules; add/ remove
Rename	✎ edit name — **per rule** (does not change other rules)
Code editor	Python `evaluate()` with syntax highlighting
BRICK targets	Multi-select from live registry + model; hidden if empty
Test rule	`POST /api/playground/test-rule` — hours from UI
Save draft	`POST /api/fdd-rules` → DynamoDB `ts_ms=-2`
Write to database	`POST /api/playground/go-live` — chunked backfill
Console	Text output from test/go-live
API load	`GET /api/fdd-rules?site_id=&building_id=` includes `brick_scope_options`

24.5 Data model

Feature	Behavior
Registry table	`GET /api/points/{site}/{building}` — all `series_id` rows from ingest
Export JSON	Open-FDD-shaped `{sites, equipment, points}`
Import JSON	`POST .../import` — preserve `metadata.external_ref` = series_id
Sync TTL	Inline Turtle projection (white panel, no popup)
LLM workflow	Export + rules → external AI → import validated JSON

24.6 System

Feature	Behavior
Health	`GET /api/health` — numpy, batch sizes, deploy revision
Log hint	`localStorage.vibe12_log=debug`

24.7 Backend tables (conceptual)

DynamoDB use	Key idea
Telemetry	`device_id` = series_id, `ts_ms` = sample time
Point registry	Meta row per site/building listing all series
Custom rules	Meta row `ts_ms=-2` JSON rules array
FDD summary	Meta row `ts_ms=0` status + flags

24.8 Browser logging policy

- **Default:** `info` — API method, path, status, duration, request id.
- **Errors:** always logged with short body snippet.
- **Not logged:** full telemetry arrays, rule source code on success.
- **Debug:** `?log=debug` or `localStorage.vibe12_log=debug`.

24.9 Build & deploy UI only

```
./scripts/build_web_ui.sh   # npm build → web_lambda/static/app
cd aws_cloud_pipeline && sam build && sam deploy
```

Related: Master checklist · Edge deploy · AWS SAM

25 Wire capture (Wireshark)

26 BACnet wire capture (pcap)

Validate RPM polling and MS/TP routing with a short **tcpdump** after each deploy.

26.1 Enable on deploy

```
cd vibe_code_apps_12/ansible
./deploy.sh --limit bacnet_pi -e enable_bacnet_read_driver=true --pcap
```

Variable	Default
`deploy_pcap_seconds`	**300** (5 minutes)
`deploy_pcap_wait_seconds`	**30**
Output file	`~/vibe_code_apps_12/captures/bacnet.pcap`

Boss Pi filter: UDP **47808** and **47809** (PiTemp + Vibe12Edge).

Ansible waits until `vibe12-bacnet-read` is **active**, then starts capture in the background as **root** (`become: true` — tcpdump requires it). Each deploy **overwrites** the same filename.

If you run the script by hand without Ansible, use **`sudo`** or capture will fail with Operation not permitted.

26.2 One command (bensserver → `$HOME`)

From the build machine (no Ansible):

```
        cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
        ./scripts/fetch_bacnet_pcap.sh              # 5 min capture +
download
        ./scripts/fetch_bacnet_pcap.sh --pull-only
# only scp existing Pi file
```

Files land at:

- `~/captures/bacnet.pcap`
- `~/bacnet-latest.pcap` (symlink)

Env: `PI_HOST=192.168.204.12 PI_USER=ben`

26.3 Pull pcap to your PC

After capture finishes, copy with an **absolute remote path**. Do **not** use `ben@host:~/vibe_code_apps_12/...` — many `scp` clients (Windows PowerShell, OpenSSH) do not expand `~` on the remote side and report No such file or directory even when the file exists.

PowerShell or bash (current directory):

```
        scp ben@192.168.204.12:/home/ben/vibe_code_apps_12/captures/
bacnet.pcap .
```

Linux/macOS (explicit home path):

```
        scp ben@192.168.204.12:/home/ben/vibe_code_apps_12/captures/
bacnet.pcap ~/bacnet.pcap
        wireshark ~/bacnet.pcap
```

Wireshark display filter: `bacnet` or `udp.port == 47808`.

26.4 Manual capture on edge

SSH to the Pi, then run with **sudo** (tcpdump needs root):

```
        sudo /home/ben/vibe_code_apps_12/scripts/bacnet_tcpdump_once.sh \
          /home/ben/vibe_code_apps_12/captures/bacnet.pcap \
          "udp port 47808 or udp port 47809" \
          120
```

Same pattern as vibe_code_apps_14/captures.

27 Commissioning backup to Git

28 Commissioning CSV backup (Git)

Device configuration for each building lives in **`commissioning/{site_id}/{building_id}/`** in the repo — not on the Pi alone.

28.1 Layout

```
commissioning/
  demo/
    bens-office/
      points.csv              # commissioned scrape list
      points_discovered.csv   # optional full discover export
      host.yml                # metadata sidecar (Ansible fetch)
```

28.2 Pull from edge after commission

```
cd vibe_code_apps_12/ansible
./fetch_commissioning.sh --limit bacnet_pi -v
```

28.3 Commit to GitHub

```
cd ~/py-bacnet-stacks-playground
git add vibe_code_apps_12/commissioning/
git commit -m "Commission BACnet points for demo/bens-office"
git push origin develop
```

Next deploy pushes `commissioning/.../points.csv` back to the edge automatically when the file exists in the repo.

28.4 Do not commit

- `aws_iot_certs/` or `ansible/files/aws_iot/*.key`
- `captures/*.pcap`

29 Cloud architecture reference

30 Vibe12 cloud — Deployed reference (BACnet MQTT)

Working stack: **`vibe12cloud`** in **`us-east-2`** — Pi BACnet read driver → MQTT → IoT rule → DynamoDB → React dashboard + Rule Lab.

Topic	Doc
Bensserver deploy	docs/aws-deploy-from-bensserver.md
CloudShell deploy	docs/aws-cloud-sam.md
Rule recipes	EXPRESSION_RULE_COOKBOOK.md

30.1 End-to-end flow

```
Raspberry Pi (boss Pi / lab)
  vibe12-bacnet-read.service  (BACpypes3 RPM poll)
        │ MQTT TLS  vibe12/{site}/{building}/{system}/{point}/
telemetry
        ▼
AWS IoT Core
        │ Rule: vibe12_telemetry_ingest  (SQL: vibe12/+/+/+/+/
telemetry)
        ▼
IngestFunction (Lambda)
        │ PutItem
        ▼
DynamoDB  vibe12-telemetry-vibe12cloud
        │
        ├──────────────────────────────┐
        ▼                              ▼
WebFunction (Lambda URL)      FddFunction (every 5 min)
  React dashboard + login       Custom rules / fdd_rules.py
```

Phase 0 lab IDs: **site** `demo`, **building** `bens-office`.

30.2 Live URLs (us-east-2, deploy revision 6+)

Output	Example
DashboardUrl	`https://mlmdwoonvb5bgltfy7dgiqv7mq0amllu.lambda-url.us-east-2.on.aws/`
Health	`{DashboardUrl}api/health`
Login	`POST {DashboardUrl}api/auth/login` JSON `username` / `password`
Readings	`GET …/api/readings?site_id=demo&building_id=bens-office` + `Authorization: Bearer …`

Login user defaults to **`engineer`** (from `samconfig.toml` `WebUsername` / `WebPassword`).

30.3 Smoke after deploy

```
export PATH="$HOME/.local/bin:$PATH"
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
chmod +x scripts/verify_cloud_dashboard.sh
./scripts/verify_cloud_dashboard.sh
```

Or manual:

```
URL="https://mlmdwoonvb5bgltfy7dgiqv7mq0amllu.lambda-url.us-east-2.on.aws"
curl -sS "${URL}/api/health"
curl -sS -X POST "${URL}/api/auth/login" -H 'Content-Type: application/json' \
  -d '{"username":"engineer","password":"<your WebPassword>"}'
```

30.4 AWS resources

Resource	Value
DynamoDB	`vibe12-telemetry-vibe12cloud`
IoT rule	`vibe12_telemetry_ingest`
MQTT pattern	`vibe12/{site_id}/{building_id}/{system_id}/{point_id}/telemetry`

Resource	Value
Lambdas runtime	**python3.12**

Delete legacy rules if still present: `vibe12_ds18b20_ingest`, `IotIngestRuleBacnet`.

30.5 Tests (local / CI)

Suite	Command
Web (Vitest)	`cd apps/vibe12-web && npm ci && npm test`
Python	`pip install -r requirements.txt -r aws_cloud_pipeline/web_lambda/requirements.txt` then `python3 -m unittest discover -s tests -v`

GitHub Actions: `.github/workflows/vibe12-tests.yml` (both jobs on push to `vibe_code_apps_12/**`).

30.6 Bensserver one-liner deploy

```
export PATH="$HOME/.local/bin:$PATH"
cd ~/py-bacnet-stacks-playground/vibe_code_apps_12
./scripts/deploy_cloud_from_bensserver.sh
```

Requires `samconfig.toml` (gitignored) with real `WebPassword` + `AuthSecret`.

30.7 Tear down

```
sam delete --stack-name vibe12cloud
```

31 FDD expression rule cookbook

32 Expression rule cookbook (Rule Lab)

Copy a recipe into **Rule Lab → Python editor**, set **Parameters (cfg)** from the table, click **Test rule** (6 h preview), then **Save draft** or **Write to database**.

32.1 How rules run

Topic	Detail
Entry point	`evaluate(row, cfg, prev_row=None, rows=None)` — required
Row fields	`row`, `ts_ms`, `ts`, `temp`, `temp_raw`, `temp_rolling_avg`, `degF`, `degC`, `degF_rolling_avg`, `sample_period_ms`, `rolling_avg_minutes`, `samples_in_avg`
Config	Use `cfg_threshold(cfg, "key")` — values follow **Rule unit** (°F or °C). Add keys via **+ Parameter** or **Add preset…**
Helpers	`temp_unit_symbol(cfg)`, `math`, `datetime`, optional `numpy as np`
Instant flag	`return True` — flags **this row only**
Retroactive lane	`return True, window_rows` — paints every row in `window_rows` on the chart (recommended for lookback rules)
Batch mode	Optional `apply_faults(rows, cfg) -> list[bool]` — see Recipe 10

Tip: At ~10 s MQTT, 6 samples ≈ 1 minute, 360 samples ≈ 1 hour. Use `FILL_RATIO = 0.95` so the window is nearly full before evaluating.

32.2 Zone starting points (DS18B20 @ ~10 s)

Recipe	Config key(s)	Start values (°F / °C)
1 — Flatline 1 h	`flatline_tolerance`	**0.10** / **0.03**
2 — Rate 1 h	`max_temp_per_hour`	**5.0** / **3.0**
3 — Rate 15 m	`max_temp_per_15min`	**2.0** / **1.1**
4 — Spread 1 h	`max_spread`	**4.0** / **2.2**
5 — Spread 15 m	`max_spread_15min`	**2.5** / **1.4**
6 — Out of bounds	`bounds_low`, `bounds_high`	**65**, **80** / **18**, **27**
7 — Flatline N samples	`flatline_tolerance`, `flatline_window`	**0.05** / **0.03**, **18**
8 — OOB debounced	`bounds_low`, `bounds_high`, `rolling_window`	**65**, **80**, **6**
9 — Rate instant (pair)	`max_temp_per_minute`	**2.0** / **1.1**
14 — Peak spread 24 h	`max_spread_24h`	**12.0** / **6.7**

32.2.1 Shipped defaults (BRICK `Zone_Air_Temperature_Sensor`)

Rule Lab loads four enabled rules from `rules_defaults.py` when no DynamoDB draft exists. Each includes `brick_scope.point_classes: ["Zone_Air_Temperature_Sensor"]` so **Test across BRICK targets** and scheduled FDD evaluate every matching zone sensor.

Default rule ID	Cookbook	What it detects
`brick_zone_oob`	Recipe 6	Comfort band violation (rolling avg)
`brick_zone_flatline_1h`	Recipe 1	Stuck sensor (1 h min–max spread)
`brick_zone_swing_15m`	Recipe 5	

Default rule ID	Cookbook	What it detects
		Short hunting / fast swing (15 m min–max)
`brick_zone_peak_swing_24h`	Recipe 14	Excessive daily peak–valley (24 h min–max)

Note: Recipes 2–3 measure **net endpoint rate** (first → last). Recipes 4–5 and 14 measure **peak swing** (max − min). Use spread recipes when a spike that returns to baseline should still fault.

Test windows: Use **≥ 2 h** for 1 h / 15 m lookback rules; use **≥ 24 h** (or max history) when testing `brick_zone_peak_swing_24h`.

32.3 Recipe 1 — Flatline (1 hour window)

Sensor stuck: max − min over the last hour is below tolerance. Paints the full hour when detected.

Config: `flatline_tolerance` = **0.10** (°F) or **0.03** (°C)

Test tip: Use **Test window ≥ 2 h** (default). `print()` runs only when a fault fires; enable **Verbose prints (window trace)** to see sample 1 h spreads without changing code.

```
ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95


def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_1_hour(row, rows)
    if len(window_rows) < 2:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False
```

```
            sym = temp_unit_symbol(cfg)
            vals = [r["temp"] for r in window_rows]
            spread = max(vals) - min(vals)
            tol = cfg_threshold(cfg, "flatline_tolerance")

            if spread < tol:
                print(
                    f"row={row['row']} ts={row['ts']} "
                    f"FLATLINE 1h spread={spread:.3f} {sym} < tol={tol:.
3f}"
                )
                return True, window_rows

            return False
```

32.4 Recipe 2 — Rate of change (1 hour, net °F/hr)

Net temperature change over a full hour, divided by elapsed hours.

Config: `max_temp_per_hour` = **5.0** (°F/hr) or **3.0** (°C/hr)

```
        ONE_HOUR_MS = 60 * 60 * 1000
        FILL_RATIO = 0.95


        def get_last_1_hour(row, rows):
            now_ms = row["ts_ms"]
            start_ms = now_ms - ONE_HOUR_MS
            return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


        def evaluate(row, cfg, prev_row=None, rows=None):
            if rows is None:
                return False

            window_rows = get_last_1_hour(row, rows)
            if len(window_rows) < 2:
                return False

            span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
            if span_ms < ONE_HOUR_MS * FILL_RATIO:
                return False

            dt_hr = (window_rows[-1]["ts_ms"] - window_rows[0]
["ts_ms"]) / 3600000.0
            if dt_hr <= 0:
                return False
```

```
    sym = temp_unit_symbol(cfg)
    v0 = window_rows[0]["temp"]
    v1 = window_rows[-1]["temp"]
    rate_hr = abs(v1 - v0) / dt_hr
    lim = cfg_threshold(cfg, "max_temp_per_hour")

    print(
        f"row={row['row']} ts={row['ts']} "
        f"rate={rate_hr:.2f} {sym}/hr ({v0:.2f} -> {v1:.2f})"
    )

    if rate_hr > lim:
        print(f"RATE/Hr: painting {len(window_rows)} rows")
        return True, window_rows

    return False
```

32.5 Recipe 3 — Rate of change (15 minutes)

Same as Recipe 2, but normalized to a 15-minute equivalent rate.

Config: `max_temp_per_15min` = **2.0** (°F) or **1.1** (°C)

```
FIFTEEN_MIN_MS = 15 * 60 * 1000
FILL_RATIO = 0.95


def get_last_15_min(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - FIFTEEN_MIN_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_15_min(row, rows)
    if len(window_rows) < 2:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < FIFTEEN_MIN_MS * FILL_RATIO:
        return False

    dt_min = (window_rows[-1]["ts_ms"] - window_rows[0]
["ts_ms"]) / 60000.0
```

```
    if dt_min <= 0:
        return False

    sym = temp_unit_symbol(cfg)
    delta = abs(window_rows[-1]["temp"] - window_rows[0]["temp"])
    rate_15 = delta * (15.0 / dt_min)
    lim = cfg_threshold(cfg, "max_temp_per_15min")

    print(
        f"row={row['row']} ts={row['ts']} "
        f"equiv {rate_15:.2f} {sym} per 15 min"
    )

    if rate_15 > lim:
        print(f"RATE/15m: painting {len(window_rows)} rows")
        return True, window_rows

    return False
```

32.6 Recipe 4 — Spread / MIN–MAX (1 hour)

Zone swung too much: peak − valley over one hour exceeds limit (short cycling, stuck damper, etc.).

Config: `max_spread` = **4.0** (°F) or **2.2** (°C)

```
ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95


def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_1_hour(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False
```

```
    sym = temp_unit_symbol(cfg)
    vals = [r["temp"] for r in window_rows]
    lo, hi = min(vals), max(vals)
    spread = hi - lo
    lim = cfg_threshold(cfg, "max_spread")

    print(
        f"row={row['row']} ts={row['ts']} "
        f"spread={spread:.2f} {sym} (min={lo:.2f} max={hi:.2f})"
    )

    if spread > lim:
        print(f"SPREAD/1h: painting {len(window_rows)} rows")
        return True, window_rows

    return False
```

32.7 Recipe 5 — Spread / MIN–MAX (15 minutes)

Short-window spread — catches fast hunting or sudden step changes.

Config: `max_spread_15min` = **2.5** (°F) or **1.4** (°C)

```
FIFTEEN_MIN_MS = 15 * 60 * 1000
FILL_RATIO = 0.95


def get_last_15_min(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - FIFTEEN_MIN_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_15_min(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < FIFTEEN_MIN_MS * FILL_RATIO:
        return False

    sym = temp_unit_symbol(cfg)
    vals = [r["temp"] for r in window_rows]
```

```
    lo, hi = min(vals), max(vals)
    spread = hi - lo
    lim = cfg_threshold(cfg, "max_spread_15min")

    print(
        f"row={row['row']} ts={row['ts']} "
        f"spread={spread:.2f} {sym} (min={lo:.2f} max={hi:.2f})"
    )

    if spread > lim:
        print(f"SPREAD/15m: painting {len(window_rows)} rows")
        return True, window_rows

    return False
```

32.8 Recipe 14 — Peak spread / MIN–MAX (24 hours)

Daily peak–valley swing: catches hunting, short cycling, or setpoint fighting that cancels out over shorter windows. Same semantics as Recipe 4–5, but over a full day. **Shipped default:** `brick_zone_peak_swing_24h`.

Config: `max_spread_24h` = **12.0** (°F) or **6.7** (°C)

Test tip: Use **Test window ≥ 24 h** (or max dashboard history). Requires ~95% fill of the 24 h window before evaluating.

```
TWENTY_FOUR_HOUR_MS = 24 * 60 * 60 * 1000
FILL_RATIO = 0.95


def get_last_24_hours(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - TWENTY_FOUR_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_24_hours(row, rows)
    if not window_rows:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < TWENTY_FOUR_HOUR_MS * FILL_RATIO:
        return False
```

```
            sym = temp_unit_symbol(cfg)
            vals = [r["temp"] for r in window_rows]
            lo, hi = min(vals), max(vals)
            spread = hi - lo
            lim = cfg_threshold(cfg, "max_spread_24h")

            print(
                f"row={row['row']} ts={row['ts']} "
                f"peak spread 24h={spread:.2f} {sym} (min={lo:.2f}
max={hi:.2f})"
            )

            if spread > lim:
                print(f"PEAK/24h: painting {len(window_rows)} rows")
                return True, window_rows

            return False
```

32.9 Recipe 6 — Out of bounds (instant, rolling avg)

Flags the current row when smoothed temperature is outside the band. Matches the shipped default **`brick_zone_oob`** rule.

Config: `bounds_low` = **65**, `bounds_high` = **80** (°F) — or **18** / **27** (°C)

Set **Rolling avg** in the toolbar to **1 min** (or **10 min** for slower response).

```
        def evaluate(row, cfg, prev_row=None, rows=None):
            sym = temp_unit_symbol(cfg)
            low = cfg_threshold(cfg, "bounds_low")
            high = cfg_threshold(cfg, "bounds_high")

            # Prefer rolling avg when present (Rule Lab computes from
toolbar setting)
            if "temp_rolling_avg" in row:
                v = row["temp_rolling_avg"]
                kind = "avg"
            elif "degF_rolling_avg" in row:
                v = row["degF_rolling_avg"]
                kind = "degF_avg"
            else:
                v = row["temp"]
                kind = "raw"

            if v < low or v > high:
                print(
```

```
                    f"{row['ts']}  OOB {kind}  {v:.2f} {sym}  "
                    f"(band {low:.1f}–{high:.1f}, raw={row['temp']:.2f})"
                )
                return True

            return False
```

32.10 Recipe 7 — Flatline (N consecutive samples)

Sample-count window (not wall-clock). Good when MQTT spacing is steady (~10 s → 18 samples ≈ 3 min).

Config: `flatline_tolerance` = **0.05**, `flatline_window` = **18**

```
        def evaluate(row, cfg, prev_row=None, rows=None):
            if rows is None:
                return False

            sym = temp_unit_symbol(cfg)
            w = int(cfg.get("flatline_window", 18))
            if w < 2:
                w = 2
            if row["row"] < w - 1:
                return False

            i = row["row"]
            win = rows[i - w + 1 : i + 1]
            vals = [r["temp"] for r in win]
            tol = cfg_threshold(cfg, "flatline_tolerance")
            spread = max(vals) - min(vals)

            if spread < tol:
                print(
                    f"{row['ts']}  FLATLINE  w={w}  spread={spread:.3f}
{sym}  "
                    f"tol={tol:.3f}"
                )
                return True, win

            return False
```

32.11 Recipe 8 — Out of bounds (debounced / sustained)

Requires **every** sample in the last N rows to be out of band before flagging (~1 min at 6 × 10 s). Reduces flicker from noise.

Config: `bounds_low` = **65**, `bounds_high` = **80**, `rolling_window` = **6**
Add `rolling_window` with **+ Parameter** (integer, default 6).

```
        def _oob(r, low, high):
            v = r.get("temp_rolling_avg", r["temp"])
            return v < low or v > high


        def evaluate(row, cfg, prev_row=None, rows=None):
            if rows is None:
                return False

            sym = temp_unit_symbol(cfg)
            low = cfg_threshold(cfg, "bounds_low")
            high = cfg_threshold(cfg, "bounds_high")
            n = int(cfg.get("rolling_window", 6))
            if n < 1:
                n = 1

            if row["row"] < n - 1:
                return False

            i = row["row"]
            win = rows[i - n + 1 : i + 1]

            if all(_oob(r, low, high) for r in win):
                v = win[-1].get("temp_rolling_avg", win[-1]["temp"])
                print(
                    f"{row['ts']}  OOB sustained {n} samples  "
                    f"last={v:.2f} {sym}  band {low:.1f}–{high:.1f}"
                )
                return True, win

            return False
```

32.12 Recipe 9 — Rate instant (previous sample pair)

Fast spike detector: rate between this row and the previous sample only. Default **Rate > limit (per minute)** style.

Config: `max_temp_per_minute` = **2.0** (°F/min) or **1.1** (°C/min)

```
        def evaluate(row, cfg, prev_row=None, rows=None):
            if not prev_row:
                return False

            sym = temp_unit_symbol(cfg)
            dt_ms = row["ts_ms"] - prev_row["ts_ms"]
            if dt_ms <= 0:
                return False

            dt_min = dt_ms / 60000.0
            rate = abs(row["temp"] - prev_row["temp"]) / dt_min
            lim = cfg_threshold(cfg, "max_temp_per_minute")

            if rate > lim:
                print(
                    f"{row['ts']}  RATE/min  {rate:.2f} {sym}/min  "
                    f"({prev_row['temp']:.2f} -> {row['temp']:.2f})"
                )
                return True

            return False
```

32.13 Recipe 10 — Verbose trace (debug / LLM report)

Prints every row when **Verbose prints** is checked in Rule Lab. Use to learn row fields or paste console into an LLM.

Config: none required

```
        def evaluate(row, cfg, prev_row=None, rows=None):
            sym = temp_unit_symbol(cfg)
            avg = row.get("temp_rolling_avg", row["temp"])
            print(
                f"#{row['row']:4d} {row['ts']}  "
                f"raw={row['temp']:.2f} avg={avg:.2f} {sym}  "
                f"samples_in_avg={row.get('samples_in_avg', '?')}"
            )
            return False
```

32.14 Recipe 11 — Batch sweep with `apply_faults` (advanced)

Runs `evaluate` once per row but merges painted windows in a second pass. Useful for heavy rules or custom merge logic.

Config: same as Recipe 2 (`max_temp_per_hour` = **5.0**)

```
ONE_HOUR_MS = 60 * 60 * 1000
FILL_RATIO = 0.95


def get_last_1_hour(row, rows):
    now_ms = row["ts_ms"]
    start_ms = now_ms - ONE_HOUR_MS
    return [r for r in rows if start_ms <= r["ts_ms"] <= now_ms]


def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False

    window_rows = get_last_1_hour(row, rows)
    if len(window_rows) < 2:
        return False

    span_ms = window_rows[-1]["ts_ms"] - window_rows[0]["ts_ms"]
    if span_ms < ONE_HOUR_MS * FILL_RATIO:
        return False

    dt_hr = span_ms / 3600000.0
    if dt_hr <= 0:
        return False

    v0 = window_rows[0]["temp"]
    v1 = window_rows[-1]["temp"]
    rate_hr = abs(v1 - v0) / dt_hr
    lim = cfg_threshold(cfg, "max_temp_per_hour")

    if rate_hr > lim:
        return True, window_rows
    return False


def apply_faults(rows, cfg):
    flags = [0] * len(rows)
    for row in rows:
        hit, window_rows = evaluate(row, cfg, rows=rows)
        if hit:
            for w in window_rows:
```

```
                    flags[w["row"]] = 1
        return flags
```

32.15 Recipe 12 — SAT–RAT spread (multi-sensor / Brick)

Cross-sensor mechanical rule using `series` context. Map aliases in rule **config**:

```
{
  "max_spread": 25.0,
  "series_aliases": {
    "SAT": "acme#tower-a#ahu-1#3456788-analog-input-2",
    "RAT": "acme#tower-a#ahu-1#3456788-analog-input-4"
  }
}
```

Or use Brick tags if series IDs match alias keys after graph import.

```
def evaluate(row, cfg, prev_row=None, rows=None, series=None):
    if not series:
        return False
    sat = series.get("SAT", {}).get("current")
    rat = series.get("RAT", {}).get("current")
    if sat is None or rat is None:
        return False
    spread = abs(float(sat) - float(rat))
    if spread > cfg["max_spread"]:
        print(f"{row['ts']}  SAT-RAT spread={spread:.1f}
SAT={sat} RAT={rat}")
        return True
    return False
```

32.16 Recipe 13 — All SAT sensors flatline (by Brick class)

Use **Test rule** with building scope after multi-series data is loaded, or query `/api/series/by-tag?brick_class=Supply_Air_Temperature_Sensor`.

```
def evaluate(row, cfg, prev_row=None, rows=None, series=None):
  # Primary row is first series in scope; use series["SAT"]
["values"] for window
  if not series or "SAT" not in series:
      return False
  vals = [v for v in series["SAT"]["values"] if v is not None]
[-18:]
```

```
            if len(vals) < 18:
                return False
            tol = cfg.get("flatline_tolerance", 0.05)
            if max(vals) - min(vals) < tol:
                print(f"{row['ts']}  SAT flatline spread={max(vals)-
min(vals):.3f}")
                return True
            return False
```

32.17 Quick workflow

1. **+ Add rule** or pick an existing rule in the dropdown.
2. Paste recipe code; add cfg keys from the table (**Add preset…**).
3. Set **Rule unit** to match your thresholds (°F vs °C).
4. **Test rule** — console shows `print` output; chart preview uses test window only.
5. **Copy report for LLM** — full rule + sweep log for debugging.
6. **Save draft** — rules only in DynamoDB (`ts_ms=-2`).
7. **Write to database** — rules + 7 d FDD backfill + status row.

32.18 Related

- Default shipped rules: `web_lambda/rules_defaults.py`
- Retroactive painting tests: `tests/test_retroactive_faults.py`
- Synthetic faults on Pi: `fault_demo_schedule.py` + `temp_sensor_server --fault-demo`